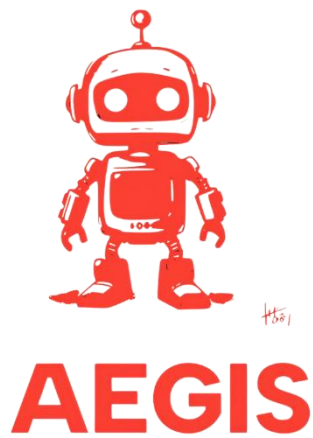


Testing Policy



Contents

Testing Objectives	3
Levels of Testing.....	3
Unit tests	3
Integration tests	3
Regression tests	3
Merge preconditions	3
Coverage	4
Ownership	4
CI enforcement.....	4
Environment	4

Testing Objectives

The primary goals we aim to achieve when testing are:

1. Ensure that requirements are met
2. Detect defects before deployment
3. Prevent the introduction of regressions when changes are made
4. Verify correct integration between components
5. Validate behaviour from an end user's perspective

Levels of Testing

Unit tests

- Testing of an individual function/component/service in isolation.
- Mock dependencies when necessary.
- Backend: uses **pytest**

Integration tests

- Verifying that multiple components work together as expected
- Sandwich integration
- Avoid usage of stubs or drivers
- Uses a job in the CI that creates an instance of the DB on every PR using a docker container
- Uses **pytest**

Regression tests

- Ensuring that new changes do not result in unintended side effects
- Automated in the CI before merging
- Uses **pytest**

Merge preconditions

Each change must be accompanied by a certain type of test

- New services/functions/components must have accompanying unit tests. These unit tests must test the expected paths as well as edge cases
- New routes should be accompanied by integration tests. The integration tests must test the success responses as well as authorisation and validation errors
- Bug fixes must be accompanied by regression tests
- Refactors must be accompanied by regression tests

Coverage

- Minimum 88% coverage on backend. This is tracked by Codecov
- Minimum 88% coverage on frontend. This is also tracked by Codecov
- A PR that drops the overall coverage noticeably is flagged by Codecov, thus preventing the merge until testing coverage is improved.

Ownership

- Whoever owns a module/service/function is responsible for its unit testing
- Parties who own individual parts should communicate when integration tests are being written
- We use Test Driven Development, so no new features can be pushed without sufficient tests

CI enforcement

The CI ensures that every check in the pipeline succeeds before a PR is allowed to be merged. The following checks are in the CI pipeline, and the following conditions causes the PR to block

- Failing tests
- Coverage drops such that Codecov flags it
- Linting errors/warnings
- SonarCloud Reliability gate

Environment

As much as the CI will run tests, each member should run through the following steps locally before making a Pull Request.

- run **pytest** in the backend
- start up an instance of **Docker** and specify the test-db URL using the CI yml file credentials. After your environment is setup, run the pytest integration command
- run **Codecov** to ensure coverage thresholds are met
- run **flake8** in the backend for linting checks
- run **ESLint** in the frontend for linting checks
- install the **SonarQube** extension to see reliability gates before merging